To: kiranjain@gmail.com, misaonakano@gmail.com

Subject: Canary cipher

Dear Kiran and Misao,

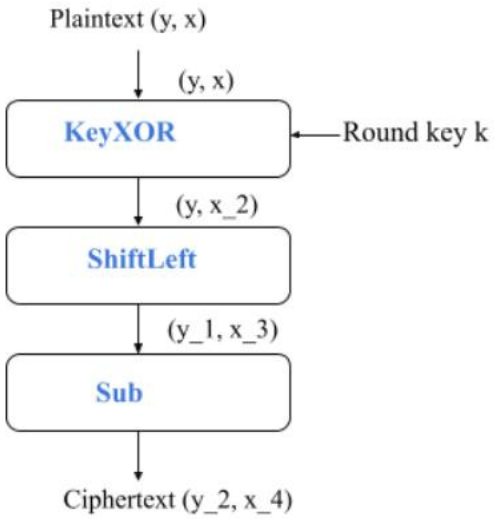
Introduction

I am writing this email to explain how encryption and decryption works in the Canary cipher. I will explain and justify the purpose of each step for the overall security using my handwritten working, block diagram, and SageMath results.

Encryption

First, I will be explaining the encryption process. Encryption is important because it plays a huge role in achieving confidentiality. Encryption prevents information that is intercepted by outsiders to be intelligible by scrambling the information. The information will be meaningless without a key that both the sender and receiver have. One round of the encryption process consists of three operations: bitwise exclusive or, 2-bit circular left shift, and 4-bit substitution.

<u>One round of operations</u>



Msg = 0xF2C5
key = 0xA5

Input : 16-bit plaintext block state (y,x) & round key k,
where :
- x is the rightmost 8-bit of the block state
- y is the leftmost 8-bit of the block state
- k is an 8-bit round key

Operation 1: KeyXOR

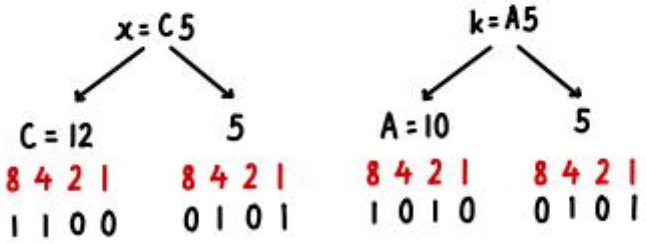
1st operation KeyXOR: ⊕ = bitwise exclusive or

(y, x) → (y, x_2)

$x_2 = x \oplus k$

XOR Truth Table		
Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

First, convert the hexadecimal values into bits. Each value in binary is raised to the powers of 2, starting from the rightmost with 2^0. Each hex value will produce four binary digits.



$x \oplus k = 1100\ 0101 \oplus 1010\ 0101$

$x_2 = 0110\ 0000 = 60_{hex}$

```
1  x = 0xC5
2  k = 0xA5
3
4  x_2 = x^^k
5
6  x_2.hex(), format(x_2, '0>8b')
```

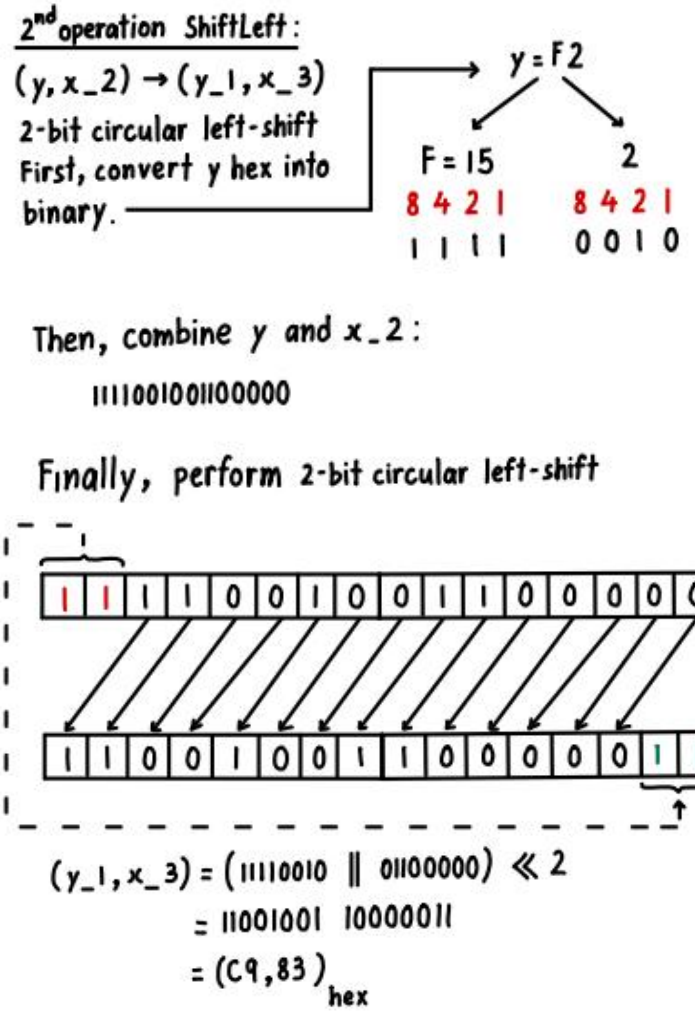
^^ = XOR

Evaluate

```
('60', '01100000')
```

As observed from the handwritten and SageMath results, the result for x_2 is 60H or 0110 0000B. This step introduces a layer of confidentiality that is key recovery (KR). This is because it is practically impossible for an attacker to obtain or guess the key.

Operation 2: ShiftLeft




```
1  y = 0xF2
2  x_2 = 0x60
3  m = 0xF260 # m combines y and x_2
4
5  m1 = ((m & 0x3FFF)<<2) + (m >> 14)
6
7  m1.hex(), format(m, '0>16b'), format(m1, '0>16b')
```

Shift left by 2-bits

Evaluate

```
('c983', '1111001001100000', '1100100110000011')
```

By shifting the digits of (y, x_2) by 2-bits, the value obtained is C983H or 1100100110000011B. This step achieves both plaintext recovery (PR) and indistinguishability (IND) as the attacker will not be able to comprehend how many bits to shift the relatively large 16-bits message.

Operation 3: Sub

3rd operation Sub:

$(y_1, x_3) \rightarrow (y_2, x_4)$

4-bit substitution on every 4-bit word of the state using the substitution table (S-box) for y_1 and x_3

$(y_1, x_3) = (C9, 83)$

$(y_2, x_4) = (C6, 49)$

Output: 16-bit ciphertext block state (y_2, x_4) where:

- rightmost 8-bit is x_4
- leftmost 8-bit is y_2

Canary S-box

Input (hex)	Output (hex)
0	7
1	F
2	0
3	9
4	3
5	1
6	D
7	B
8	4
9	6
A	2
B	8
C	C
D	E
E	5
F	A

```
m = 0xC983 # m is the combination of y_1 and x_3
n1 = int(m.hex()[0], 16)
n2 = int(m.hex()[1], 16)
n3 = int(m.hex()[2], 16)
n4 = int(m.hex()[3], 16)

Sbox = ([0x7, 0xF, 0x0, 0x9, 0x3, 0x1, 0xD, 0xB, 0x4, 0x6, 0x2, 0x8, 0xC, 0xE, 0x5, 0xA])
o1 = Sbox[n1]
o2 = Sbox[n2]
o3 = Sbox[n3]
o4 = Sbox[n4]
format(o1, '0>1x'), format(o2, '0>1x'), format(o3, '0>1x'), format(o4, '0>1x')
```

Index is 0, 1, …, F. (e.g. 0 = 7, 1 = F, 15 = A)

Evaluate

```
('c', '6', '4', '9')
```

Utilizing the Substitution table (S-box), the inputs are compared to their corresponding outputs to produce the result. (y_2, x_4) = C649H or

1100011001001001B. This step also achieves indistinguishability (IND) as the attacker would not know how the substitution table is organised.

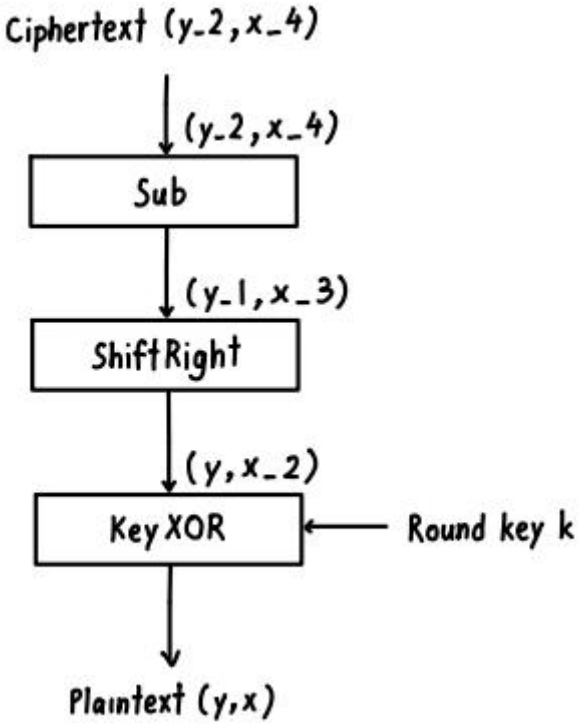
Encryption Conclusion

With the initial message of (y, x) = F2C5H and the key = A5H, the message has been encrypted to (y_2, x_4) = C649H. Through these operations with layers of confidentiality, it would be difficult for an attacker to easily intercept the message.

Decryption

Now that the message is encrypted, it will be difficult for outsiders to intercept the message. The receiver, however, with the same key and knowledge of the operations as the sender, can reverse the encryption process to decrypt the message.

One round of operations



Operation 1: Sub

```
m = 0xC649 # m is the combination of y_2 and x_4
n1 = int(m.hex()[0], 16)
n2 = int(m.hex()[1], 16)
n3 = int(m.hex()[2], 16)
n4 = int(m.hex()[3], 16)

Sbox = ([0x2, 0x5, 0xA, 0x4, 0x8, 0xE, 0x9, 0x0, 0xB, 0x3, 0xF, 0x7, 0xC, 0x6, 0xD, 0x1])
o1 = Sbox[n1]
o2 = Sbox[n2]
o3 = Sbox[n3]
o4 = Sbox[n4]
format(o1, '0>1x'), format(o2, '0>1x'), format(o3, '0>1x'), format(o4, '0>1x')
```

The inputs and outputs of the S-box are swapped

e.g. ShiftLeft: 0 = 7, 14 = 5 | ShiftRight: 7 = 0, 5 = E

Evaluate

```
('c', '9', '8', '3')
```

Utilizing the same table as before but swapping the input and outputs, the encrypted message goes through the same process in reverse to obtain C983H.

Operation 2: ShiftRight

```
m = 0xC983

m1 = (m >> 2) + (m << 14)
m2 = m1 & 0xFFFF

m2.hex(), format(m, '0>16b'), format(m2, '0>16b')
```

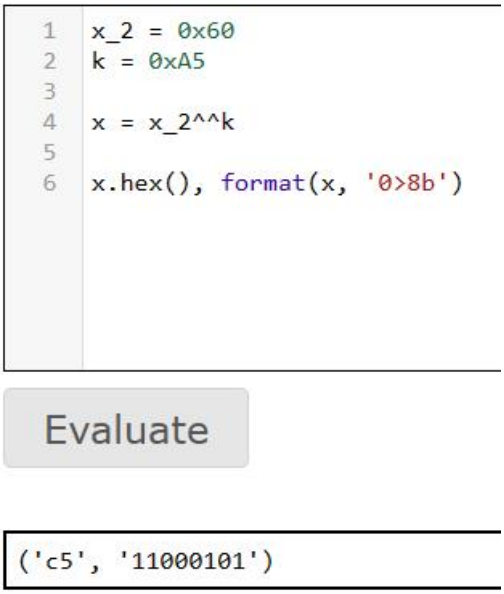
Shift right by 2-bits

Evaluate

```
('f260', '1100100110000011', '1111001001100000')
```

By shifting the digits to the right by 2-bits. (y, x_2) = F260H can be obtained.

Operation 3: KeyXOR



```
x_2 = 0x60
k = 0xA5

x = x_2^^k

x.hex(), format(x, '0>8b')
```

Evaluate

```
('c5', '11000101')
```

By using the same key as the sender, x_2 XOR k will result in x = C5.

Decryption Conclusion

After going through the operations with the same key as the sender, A5H, the encrypted message, C649H, has been decrypted to the original message, F2C5H.

Conclusion

To summarise, the Canary cipher utilises a key and undergoes three operations: bitwise exclusive or, 2-bit circular left shift, and 4-bit substitution, to encrypt messages. Only by knowing the key and the method to reverse the operations, can the encrypted message be decrypted to reveal the original message. I hope you find this email helpful in understanding how the Canary cipher works.

Regards,

Braden